WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

仓颉并发机制的设计与实现分享

主讲人：陈冬杰



目录

01 背景

02 仓颉并发机制介绍

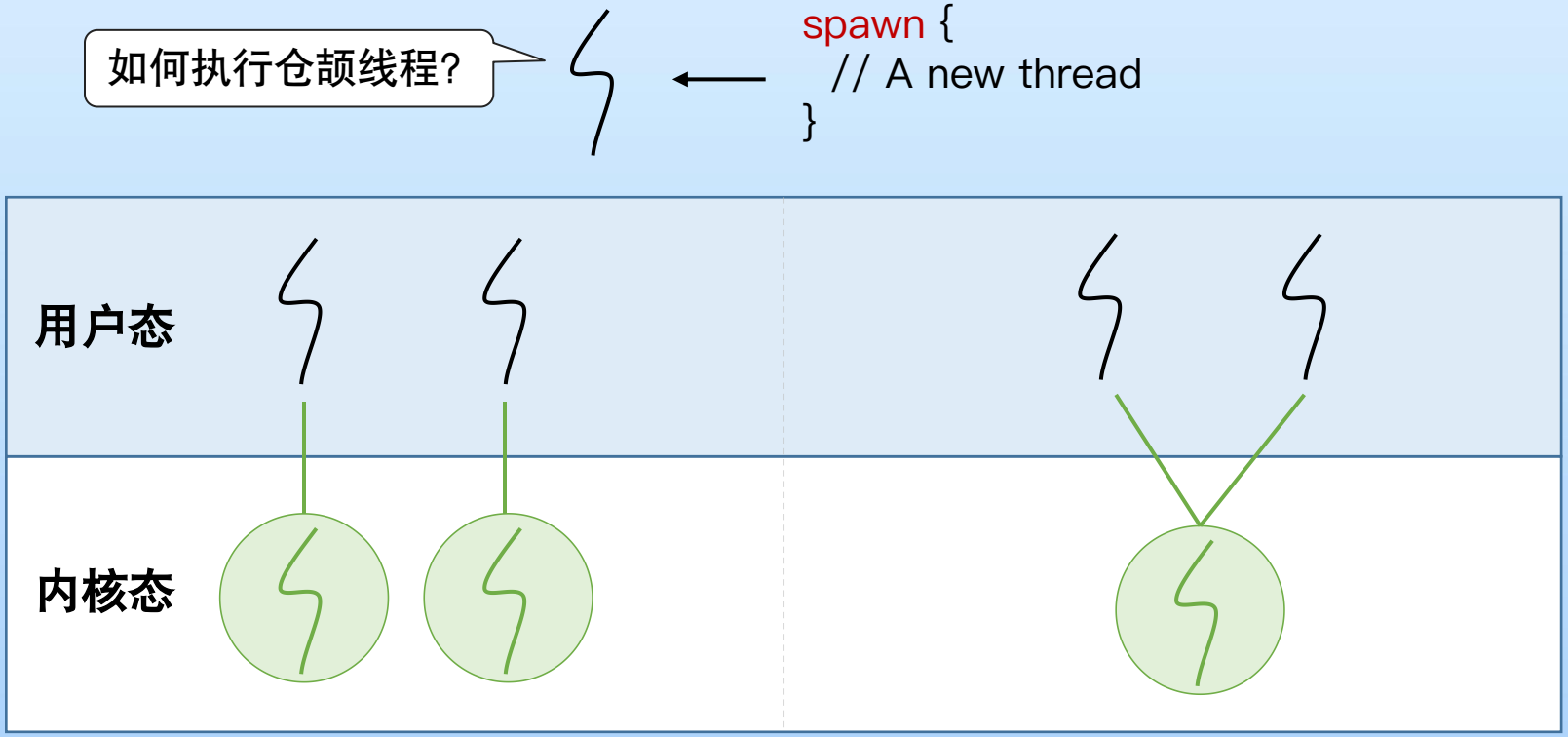
03 小结

背景

仓颉语言使用 用户态 有栈协程 模型

- 内核态 VS. 用户态
- 有栈 VS. 无栈

内核态线程 VS. 用户态线程



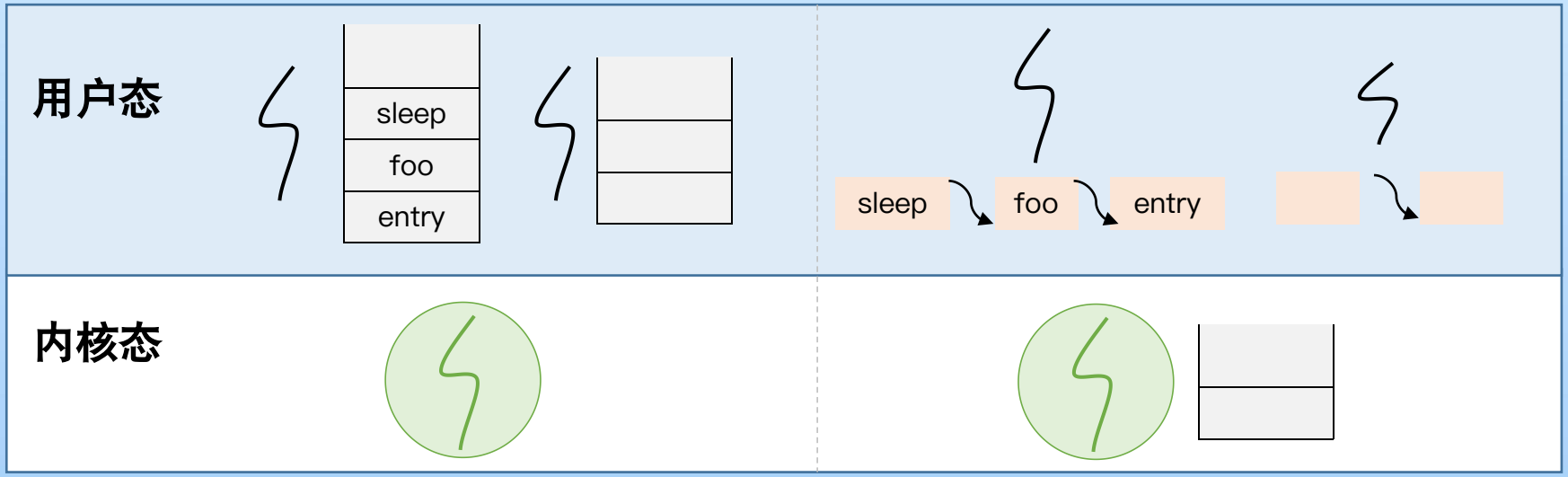
内核线程方案：仓颉线程与内核线程一一对应
- 实现简单
- 运行时开销大
- 最大线程数量受限

用户态线程方案：仓颉线程与内核线程多对多关联
- 实现复杂
- 运行时开销小
- 最大线程数 >> 内核线程数

有栈 VS. 无栈（用户态线程）

```
func foo() { sleep() }
spawn {
  // ...
  foo()
  // ...
}
```

```
async func foo() { sleep() }
spawn {
  // ...
  await sleep()
  // ...
}
```



有栈方案：用户态线程有独立的执行栈

- 开发者使用更简洁（避免关键字传播问题[1]）

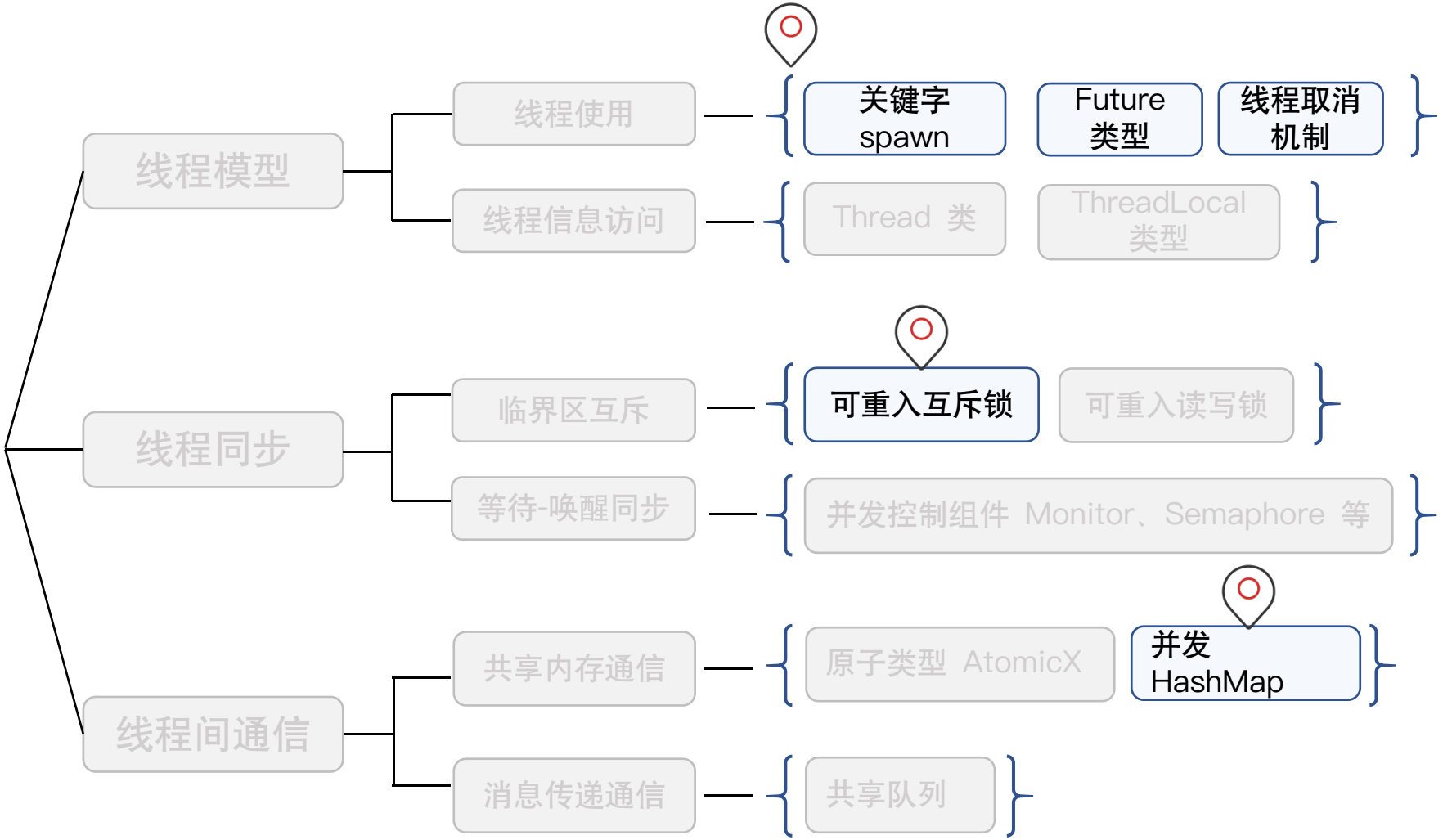
无栈方案：用户态线程使用链表管理执行信息，借助内核线程的栈完成执行

- 能够“压榨出”更极致的性能

1：关键字async具有传染性，调用async的函数也必须是async定义。

仓颉 Cangjie

仓颉并发机制介绍



本次报告涉及的分享内容

仓颉线程使用示例

2. 类型 Future<T> 包装了线程的执行结果

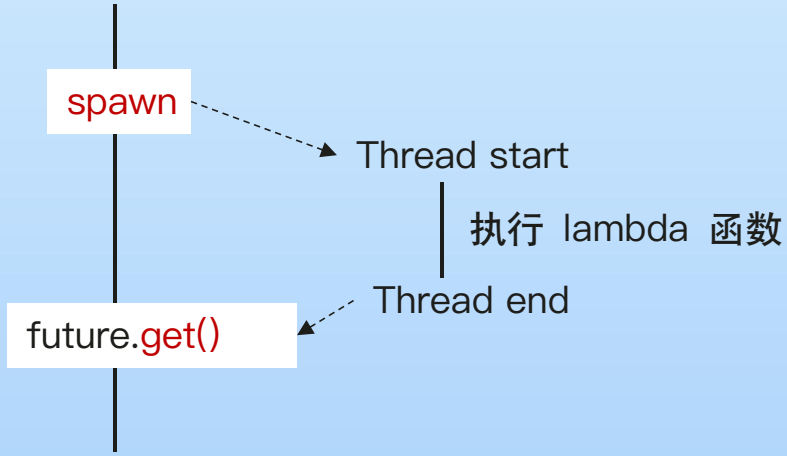
1. 关键字 spawn 尾随 lambda 函数，新建仓颉线程并返回 Future 对象

```
let future: Future<String> = spawn {
  return “Cangjie”
}

let result: String = future.get()

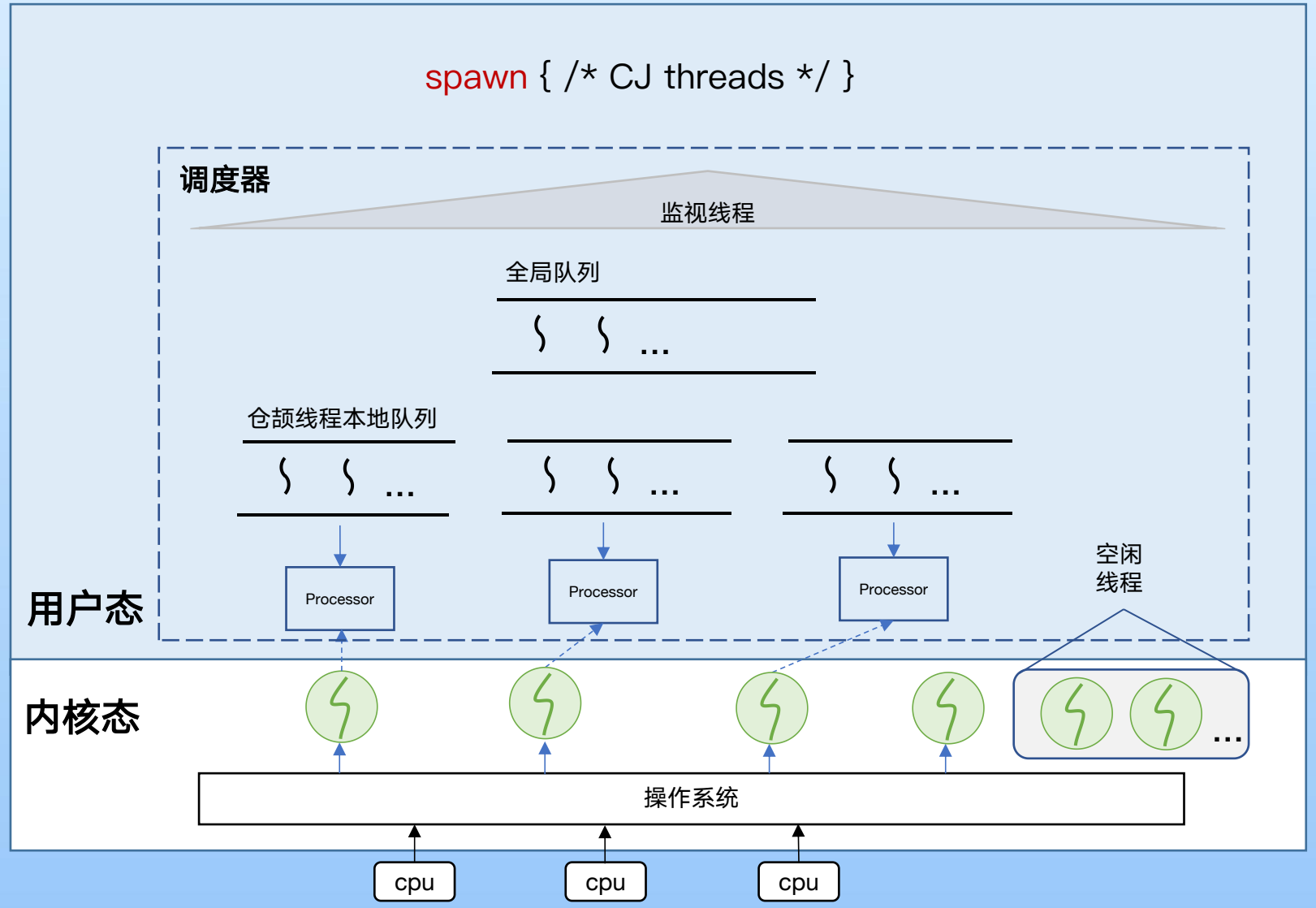
println(“hello ${result}”)
```

3. Future<T> 的 get() 方法等待线程的执行结束并获取实际结果



执行时序图示

仓颉线程实现概览


spawn { /* CJ threads */ }
调度器
监视线程
全局队列
仓颉线程本地队列
Processor
Processor
Processor
空闲
线程
用户态
内核态
操作系统
cpu
cpu
cpu

线程取消机制使用示例

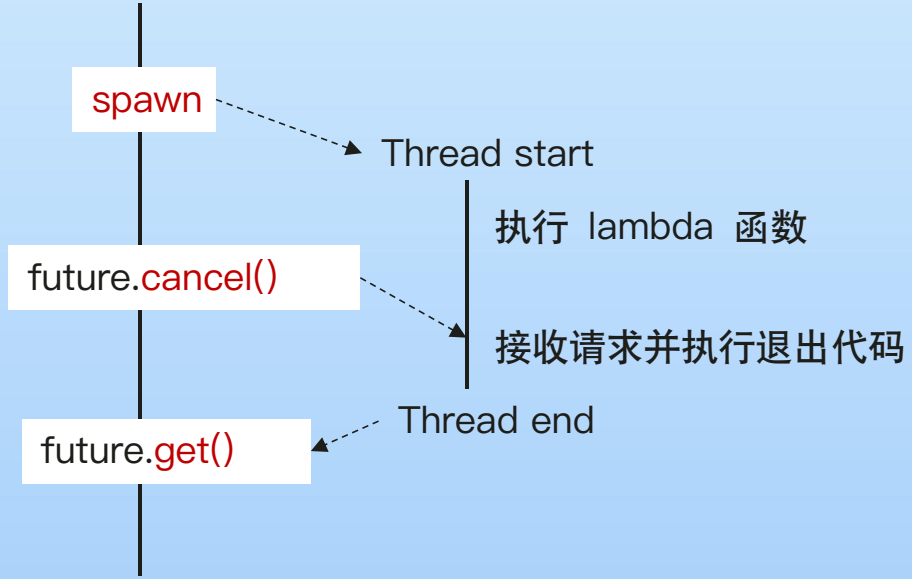
关键点：线程的取消处理应该完全交由业务代码管理，才能正确地在线程退出前释放资源

```
let future = spawn {
  …
  if (Thread.currentThread.hasPendingCancellation) {
    return …
  }
  …
}

future.cancel()
future.get()
```

该属性检查线程存在取消请求；
业务代码检查并完成退出

仅发送取消请求



执行时序图示

协作式线程取消设计

自动取消方案

```
let future = spawn {
  let f = File(…)
  process(f)
  f.close()
}

…
future.kill()
```

线程此处自动终止

文件未关闭，资源泄露！

在预设的程序点上自动终止线程
- 使用简单
- 易导致程序错误

强制终止一个线程已在实践中被反复证明是一个不良设计

协作式取消方案

```
let future = spawn {
  let f = File(…)
  process(f)
  if (Thread.currentThread.hasPendingCancellation) {
    …
  }
 f.close()
}

future.cancel()
```

关键点：线程取消处理应完全交由业务代码管理，才能正确地在线程退出前释放资源

线程取消处理交由开发者管理
- 使用繁琐
- 安全！

仓颉 Cangjie

可重入互斥锁使用示例


```
let m = ReentrantMutex()
func foo() {
  synchronized(m) { bar() }
}
func bar() {
  synchronized(m) { ... }
}
```

可重入性帮助开发者减少引入死锁风险

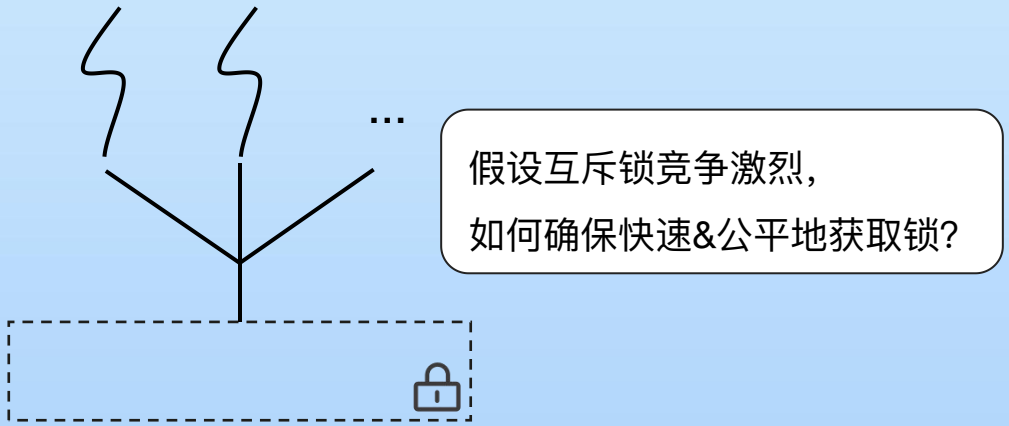
synchronized 关键字简化互斥锁的使用，减少漏释放锁的风险

```
var a = 0
let mutex = ReentrantMutex()
spawn {
  synchronized(mutex) {
    ...
    a += 1
  }
}
```

临界区保护，仅有唯一线程可进入执行

仓颉 Cangjie

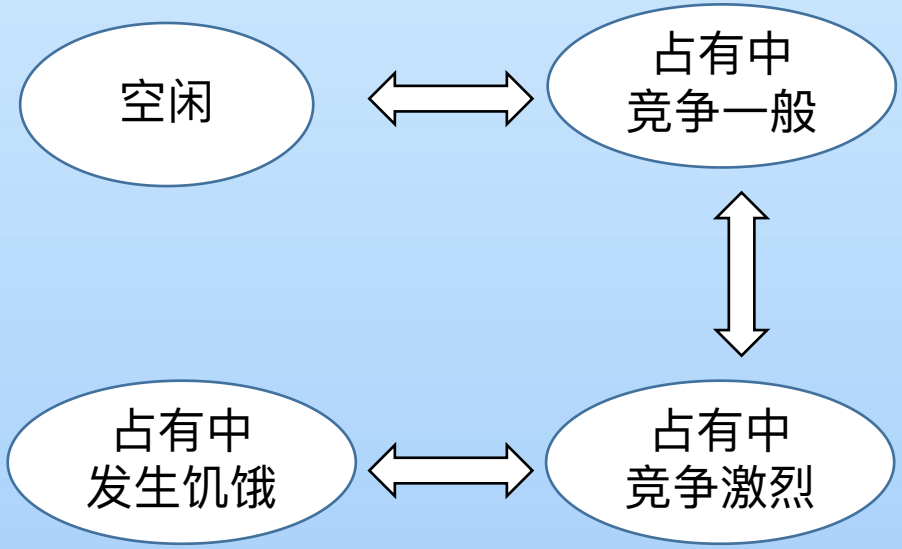
可重入互斥锁实现概览



互斥锁被设计为状态机，在每个状态下使用不同的锁获取方式，以达到更高效率

一条 CAS 指令即可获得锁，
实现快速路径

自旋方式获取锁，
加快锁获取效率



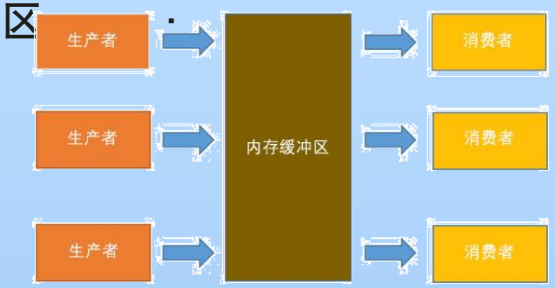
饥饿线程优先获取锁，
保证公平性

进入等待队列依次获取锁，
减少竞争

仓颉 Cangjie

并发数据结构使用示例

- 应用场景：并发数据结构在多线程系统中很常见
  - 并发哈希表：实现数据库缓存（cache）···
  - 并发队列：作为多线程通讯的缓冲区：



使用互斥锁：在任意程序点使用哈希表，开发者都需要手动添加互斥锁保护，大大增加编程复杂度

```
let connections: HashMap<String, Client>
let mutex: ReentrantMutex

func record(ip: String, client: Client) {
  synchronized(mutex) {
    connections.putIfAbsent(ip, client)
  }
}
```

使用并发哈希表：并发数据结构的内部实现保证了方法执行具有原子性，无需互斥锁

```
let connections: ConcurrentHashMap<String, Client>

func record (ip: String, client: Client) {
  connections.putIfAbsent(ip, client)
}
```

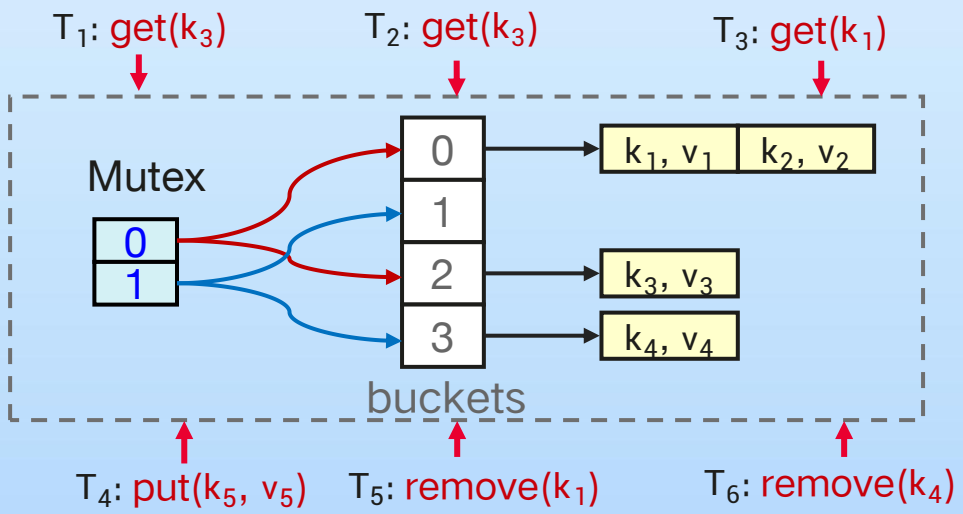

仓颉 Cangjie

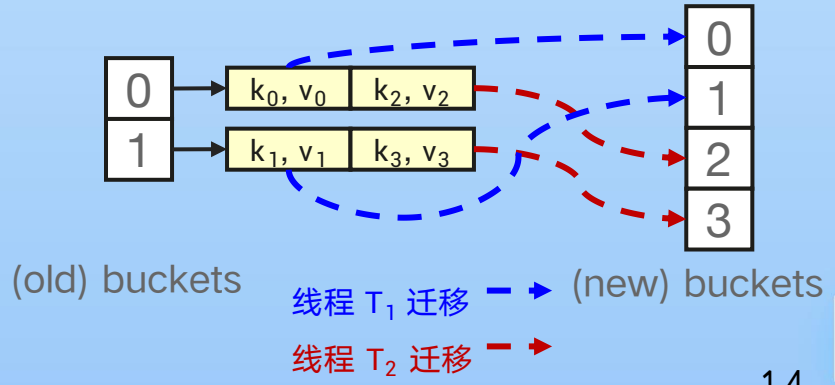
并发哈希表实现概览

仓颉 ConcurrentHashMap 使用细粒度并发算法实现

- 查询表数据操作（get）可并行
  - 线程 T_1、T_2 和 T_3 并发执行不会互相阻塞
- 修改（put，remove 等）和查询表数据可并行，例如：
  - 线程 T_1 和 T_4 并发执行无需同步
- 修改不同 Mutex 保护的 Bucket 无需同步，例如：
  - 线程 T_5 和 T_6 并发执行需同步
- 修改相同 Mutex 保护的 Bucket 需要同步，例如：
  - 线程 T_4（假设 k_5 插入 bucket[1]）和 T_6 并发执行需同步

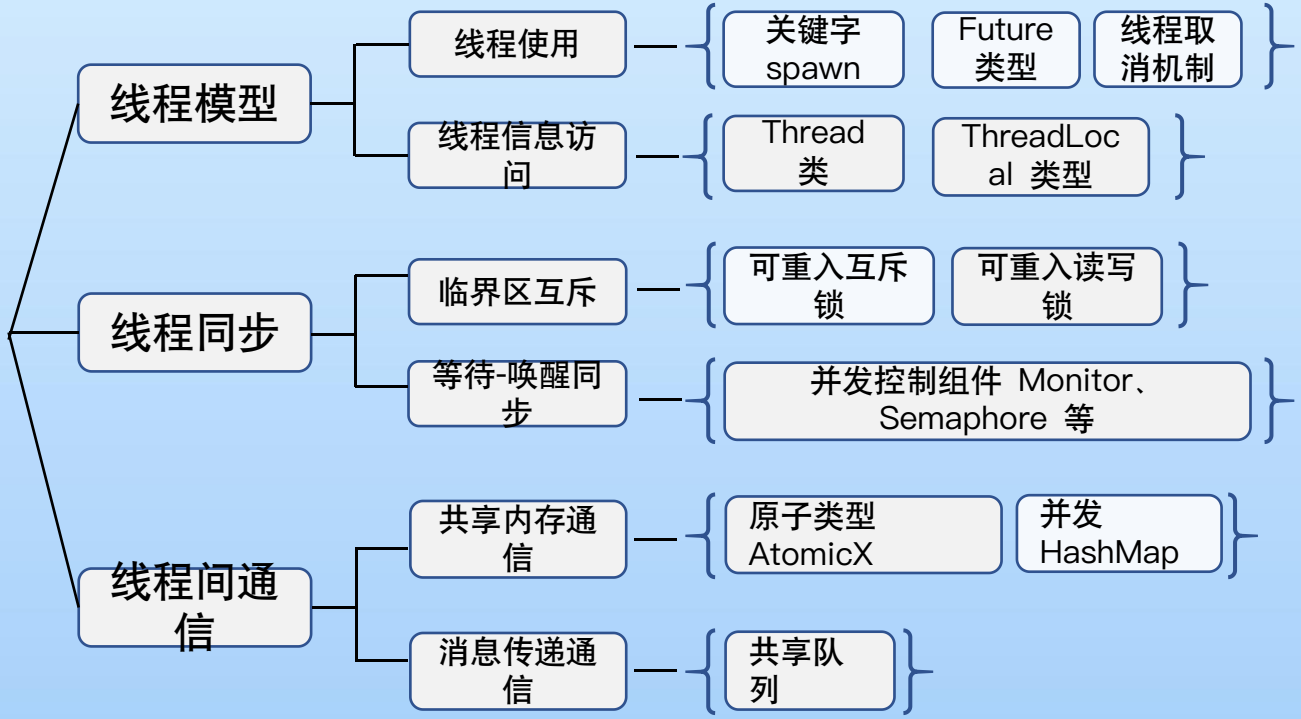
多线程数据迁移：不同线程负责不同 bucket 中键值对的迁移





小结

- 仓颉语言使用 用户态 有栈协程 模型
- 为开发者提供完善的并发编程机制
- 采用高性能、高可用的实现方案



仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，获取最新官方资讯!
(您也可以替换为自己的二维码/联系方式，邀请社区开发者共建项目)

